

Skill Session 13

Task 1

To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

```
j_rakshitha@RAKSHITHA:~$ cd ~/OSSP
echo "Hello from input redirection!" > input.txt
echo "This is Session 13 Task 1." >> input.txt
j_rakshitha@RAKSHITHA:~/OSSP$ cat input.txt
Hello from input redirection!
This is Session 13 Task 1.
```

```
j_rakshitha@RAKSHITHA:~/OSSP$ nano session13_input.c
j_rakshitha@RAKSHITHA:~/OSSP$ gcc -Wall -Wextra -g session13_input.c -o session13_input
j_rakshitha@RAKSHITHA:~/OSSP$ ./session13_input
Session 13 - Input Redirection
Enter command using <, for example:
cat < input.txt

input-shell$ cat < input.txt
Hello from input redirection!
This is Session 13 Task 1.

Input redirection completed successfully.
j_rakshitha@RAKSHITHA:~/OSSP$ |
```

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char command[256];

    printf("Session 13 - Input Redirection\n");
    printf("Enter command using <, for example:\n");
    printf("cat < input.txt\n\n");

    printf("input-shell$ ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        return 1;

    command[strcspn(command, "\n")] = '\0';

    char *redirect = strchr(command, '<');

    if (redirect == NULL)
    {
        printf("Error: no input redirection operator found.\n");
        return 1;
    }

    *redirect = '\0';

    char *program = command;
    char *filename = redirect + 1;

    /* Remove leading spaces from program */
    while (*program == ' ')
        program++;

    /* Remove trailing spaces from program */
```

GNU nano 8.7.1

```
int program_length = strlen(program);

while (program_length > 0 &&
       program[program_length - 1] == ' ')
{
    program[program_length - 1] = '\0';
    program_length--;
}

/* Remove leading spaces from filename */
while (*filename == ' ')
    filename++;

/* Remove trailing spaces from filename */
int filename_length = strlen(filename);

while (filename_length > 0 &&
       filename[filename_length - 1] == ' ')
{
    filename[filename_length - 1] = '\0';
    filename_length--;
}

if (*program == '\0')
{
    printf("Error: command is missing.\n");
    return 1;
}

if (*filename == '\0')
{
    printf("Error: input file is missing.\n");
    return 1;
}

/* Open input file */
int fd = open(filename, O_RDONLY);

if (fd == -1)
{
    perror("open input file");
```

GNU nano 8.7.1

```
        return 1;
    }

    /* Save original stdin */
    int saved_stdin = dup(STDIN_FILENO);

    if (saved_stdin == -1)
    {
        perror("dup");
        close(fd);
        return 1;
    }

    /* Redirect stdin to the input file */
    if (dup2(fd, STDIN_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        close(saved_stdin);
        return 1;
    }

    close(fd);

    /* Create child process */
    pid_t pid = fork();

    if (pid == -1)
    {
        perror("fork");
        dup2(saved_stdin, STDIN_FILENO);
        close(saved_stdin);
        return 1;
    }
}
```

```
j_rakshitha@RAKSHITHA: ~/C × + v
GNU nano 8.7.1
    close(fd);
    close(saved_stdin);
    return 1;
}

close(fd);

/* Create child process */
pid_t pid = fork();

if (pid == -1)
{
    perror("fork");
    dup2(saved_stdin, STDIN_FILENO);
    close(saved_stdin);
    return 1;
}

if (pid == 0)
{
    /* Execute command */
    execlp(program, program, (char *)NULL);

    perror("exec");
    exit(1);
}

/* Parent waits for child */
waitpid(pid, NULL, 0);

/* Restore stdin */
if (dup2(saved_stdin, STDIN_FILENO) == -1)
    perror("restore stdin");

close(saved_stdin);

printf("\nInput redirection completed successfully.\n");

return 0;
}
```

Task 2

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

```
j_rakshitha@RAKSHITHA: ~  
j_rakshitha@RAKSHITHA:~$ nano session13_task2_output.c  
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g session13_task2_output.c -o session13_task2_output  
j_rakshitha@RAKSHITHA:~$ ./session13_task2_output  
Session 13 - Task 2: Output Redirection  
Example: echo Hello > output.txt  
  
output-shell$ echo Hello_OSSP > output_task2.txt  
Output redirection completed successfully.  
j_rakshitha@RAKSHITHA:~$ cat output_task2.txt  
Hello_OSSP  
j_rakshitha@RAKSHITHA:~$ ./session13_task2_output  
Session 13 - Task 2: Output Redirection  
Example: echo Hello > output.txt  
  
output-shell$ echo Second_Message > output_task2.txt  
Output redirection completed successfully.  
j_rakshitha@RAKSHITHA:~$ cat output_task2.txt  
Second_Message  
j_rakshitha@RAKSHITHA:~$
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session13_task2_output.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main()
{
    char command[256];

    printf("Session 13 - Task 2: Output Redirection\n");
    printf("Example: echo Hello > output.txt\n\n");

    printf("output-shell$ ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        return 1;

    command[strcspn(command, "\n")] = '\0';

    char *redirect = strchr(command, '>');

    if (redirect == NULL)
    {
        printf("Error: '>' operator not found.\n");
        return 1;
    }

    *redirect = '\0';

    char *program = command;
    char *filename = redirect + 1;

    while (*program == ' ' || *program == '\t')
        program++;

    while (*filename == ' ' || *filename == '\t')
        filename++;

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session13_task2_output.c

int len = strlen(program);
while (len > 0 && (program[len - 1] == ' ' || program[len - 1] == '\t'))
{
    program[len - 1] = '\0';
    len--;
}

len = strlen(filename);
while (len > 0 && (filename[len - 1] == ' ' || filename[len - 1] == '\t'))
{
    filename[len - 1] = '\0';
    len--;
}

int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

if (fd == -1)
{
    perror("open");
    return 1;
}

pid_t pid = fork();

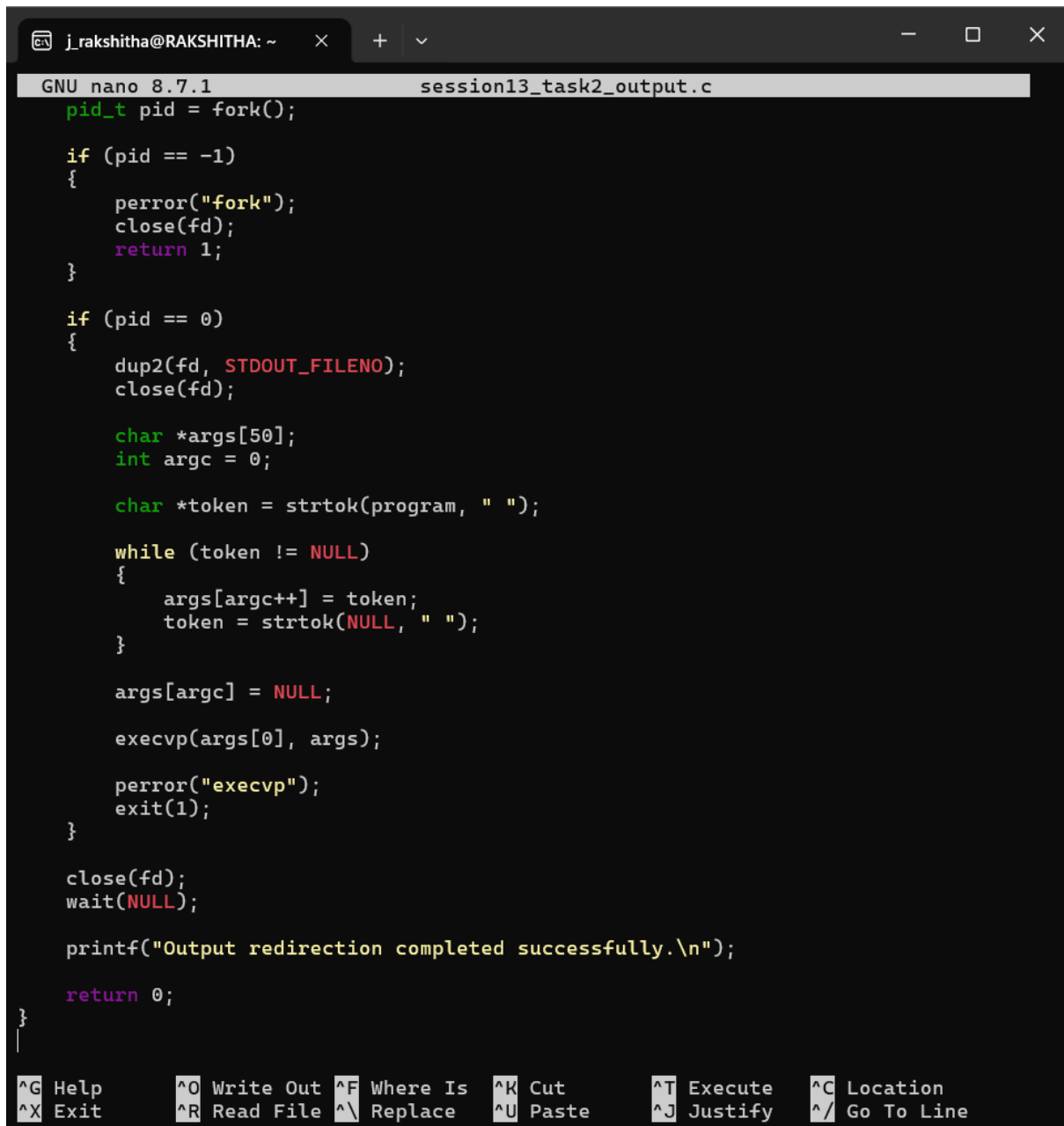
if (pid == -1)
{
    perror("fork");
    close(fd);
    return 1;
}

if (pid == 0)
{
    dup2(fd, STDOUT_FILENO);
    close(fd);

    char *args[50];
    int argc = 0;

    char *token = strtok(program, " ");

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```



The screenshot shows a terminal window with a dark background. At the top, the window title bar displays 'j_rakshitha@RAKSHITHA: ~' and standard window controls. Below the title bar, the nano editor's status line shows 'GNU nano 8.7.1' and the filename 'session13_task2_output.c'. The main area contains C code for a program that forks a child process, sets up arguments, and executes a command. The code is as follows:

```
pid_t pid = fork();

if (pid == -1)
{
    perror("fork");
    close(fd);
    return 1;
}

if (pid == 0)
{
    dup2(fd, STDOUT_FILENO);
    close(fd);

    char *args[50];
    int argc = 0;

    char *token = strtok(program, " ");

    while (token != NULL)
    {
        args[argc++] = token;
        token = strtok(NULL, " ");
    }

    args[argc] = NULL;

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

close(fd);
wait(NULL);

printf("Output redirection completed successfully.\n");

return 0;
}
```

At the bottom of the terminal, a row of keyboard shortcuts is displayed:

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^I Replace	^U Paste	^J Justify	^_ Go To Line

Skill Session 14

Task 1

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios

```
j_rakshitha@RAKSHITHA:~$ nano session14_task1_append.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g session14_task1_append.c -o session14_task1_append
j_rakshitha@RAKSHITHA:~$ ./session14_task1_append
Session 14 - Task 1: Append Redirection
Example: echo Hello >> output.txt

append-shell$ echo First_Line >> append_test.txt
Append redirection completed successfully.
j_rakshitha@RAKSHITHA:~$ cat append_test.txt
First_Line
j_rakshitha@RAKSHITHA:~$ ./session14_task1_append
Session 14 - Task 1: Append Redirection
Example: echo Hello >> output.txt

append-shell$ echo Second_Line >> append_test.txt
Append redirection completed successfully.
j_rakshitha@RAKSHITHA:~$ cat append_test.txt
First_Line
Second_Line
j_rakshitha@RAKSHITHA:~$ |
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session14_task1_append.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    char command[256];

    printf("Session 14 - Task 1: Append Redirection\n");
    printf("Example: echo Hello >> output.txt\n\n");

    printf("append-shell$ ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
    {
        return 1;
    }

    char *operator = strstr(command, ">>");

    if (operator == NULL)
    {
        printf("Error: '>>' operator not found.\n");
        return 1;
    }

    *operator = '\0';

    char *filename = operator + 2;

    /* Remove newline */
    filename[strlen(filename, "\n")] = '\0';

    /* Remove leading spaces from filename */
    while (*filename == ' ' || *filename == '\t')
    {
        [ Read 136 lines ]
    }
}

^G Help      ^O Write Out  ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File  ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session14_task1_append.c
    filename++;
}

/* Remove trailing spaces */
char *end = filename + strlen(filename) - 1;
while (end >= filename && (*end == ' ' || *end == '\t'))
{
    *end = '\0';
    end--;
}

/* Remove leading spaces from command */
char *program = command;
while (*program == ' ' || *program == '\t')
{
    program++;
}

/* Remove trailing spaces */
end = program + strlen(program) - 1;
while (end >= program && (*end == ' ' || *end == '\t'))
{
    *end = '\0';
    end--;
}

if (strlen(program) == 0 || strlen(filename) == 0)
{
    printf("Error: Invalid command or filename.\n");
    return 1;
}

/*
 * O_APPEND means new data is added to the end
 * instead of replacing existing data.
 */
int fd = open(filename, O_WRONLY | O_CREAT | O_APPEND, 0644);

if (fd == -1)
{
    perror("open");
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session14_task1_append.c
    return 1;
}

pid_t pid = fork();

if (pid == -1)
{
    perror("fork");
    close(fd);
    return 1;
}

if (pid == 0)
{
    /* Redirect stdout to the file */
    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        exit(1);
    }

    close(fd);

    /* Convert command into arguments */
    char *args[50];
    int count = 0;
```

```
    char *token = strtok(program, " \\t");

    while (token != NULL && count < 49)
    {
        args[count++] = token;
        token = strtok(NULL, " \\t");
    }

    args[count] = NULL;

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}
else
{
    close(fd);

    waitpid(pid, NULL, 0);

    printf("Append redirection completed successfully.\n");
}

return 0;
}
```

Task 2

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

```

j_rakshitha@RAKSHITHA:~$ nano session14_task2_stderr.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -g session14_task2_stderr.c -o session14_task2_stderr
j_rakshitha@RAKSHITHA:~$ ./session14_task2_stderr
Session 14 - Task 2: stderr Redirection
Example: ls missing_file 2> error.txt

stderr-shell$ ls missing_file 2> error.txt
stderr redirection completed successfully.
j_rakshitha@RAKSHITHA:~$ cat error.txt
ls: cannot access 'missing_file': No such file or directory
j_rakshitha@RAKSHITHA:~$ ./session14_task2_stderr
Session 14 - Task 2: stderr Redirection
Example: ls missing_file 2> error.txt

stderr-shell$ badcommand 2> error.txt
stderr redirection completed successfully.
j_rakshitha@RAKSHITHA:~$ cat error.txt
execvp: No such file or directory
j_rakshitha@RAKSHITHA:~$

```

```

GNU nano 8.7.1 session14_task2_stderr.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    char command[256];

    printf("Session 14 - Task 2: stderr Redirection\n");
    printf("Example: ls missing_file 2> error.txt\n\n");

    printf("stderr-shell$ ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
    {
        return 1;
    }

    /* Find the 2> operator */
    char *operator = strstr(command, "2>");

    if (operator == NULL)
    {
        printf("Error: '2>' operator not found.\n");
        return 1;
    }

    /* Separate command and filename */
    *operator = '\0';

    char *filename = operator + 2;

    /* Remove newline */
    filename[strcspn(filename, "\n")] = '\0';

    /* Remove leading spaces from filename */
    while (*filename == ' ' || *filename == '\t')
    {
        filename++;
    }

    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line

```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 session14_task2_stderr.c
/* Remove trailing spaces from filename */
char *end = filename + strlen(filename) - 1;

while (end >= filename && (*end == ' ' || *end == '\t'))
{
    *end = '\0';
    end--;
}

/* Remove leading spaces from command */
char *program = command;

while (*program == ' ' || *program == '\t')
{
    program++;
}

/* Remove trailing spaces from command */
end = program + strlen(program) - 1;

while (end >= program && (*end == ' ' || *end == '\t'))
{
    *end = '\0';
    end--;
}

if (strlen(program) == 0 || strlen(filename) == 0)
{
    printf("Error: Invalid command or filename.\n");
    return 1;
}

/* Open/create the error file */
int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

if (fd == -1)
{
    perror("open");
    return 1;
}

pid_t pid = fork();

if (pid == -1)
{
    |

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify	^_ Go To Line

```
j_rakshitha@RAKSHITHA: ~ × + v
GNU nano 8.7.1 session14_task2_stderr.c

pid_t pid = fork();

if (pid == -1)
{
    perror("fork");
    close(fd);
    return 1;
}

if (pid == 0)
{
    /* Redirect stderr to the file */
    if (dup2(fd, STDERR_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        exit(1);
    }

    close(fd);

    /* Convert command into arguments */
    char *args[50];
    int count = 0;

    char *token = strtok(program, " \\t");

    while (token != NULL && count < 49)
    {
        args[count++] = token;
        token = strtok(NULL, " \\t");
    }

    args[count] = NULL;

    /* Execute command */
    execvp(args[0], args);

    /* This error also goes into the redirected stderr file */
    perror("execvp");
    exit(1);
}
else
{
    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
    ^X Exit      ^R Read File  ^\\ Replace    ^U Paste      ^J Justify    ^_ Go To Line

```

```

    {
        close(fd);

        waitpid(pid, NULL, 0);

        printf("stderr redirection completed successfully.\n");
    }

    return 0;
}
|
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\\ Replace    ^U Paste      ^J Justify    ^_ Go To Line

```